

21720 Programació concurrent

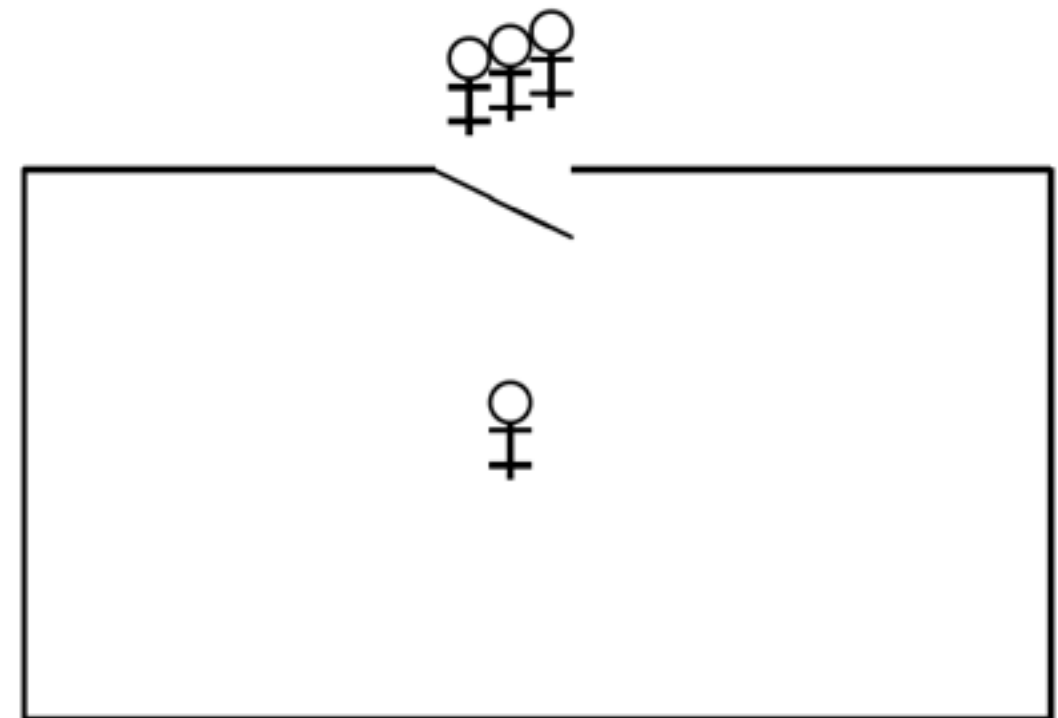
2. El problema de la regió crítica

Introducció al problema

- Exemple genèric de la cua d'impressió
- Solucions com l'algorisme de Dekker són poc usades a la pràctica ja que els sistemes tenen mecanismes de sincronització més eficients
- L'estudi d'aquests algorismes incorrectes demostra les patologies típiques dels algorismes concurrents

Definició del problema

- N processos executen en un bucle infinit una seqüència d'instruccions que poden ser dividides en dues subsequències: la regió crítica i la regió no crítica
- Requisits per a qualsevol solució:
 - **Exclusió mútua**: Instruccions de la regió crítica de dos o més processos no poden ser intercalades
 - **No hi ha interbloquejos** (*deadlock free*): Si alguns processos intenten entrar a la regió crítica, un d'ells ho ha d'aconseguir
 - **Espera limitada** (no hi ha inanició *starvation free*): Qualsevol procés que vulgui entrar a la regió crítica ho ha d'aconseguir en un temps finit



- Mecanisme de sincronització per a garantir la correctesa dels requisits: Instruccions addicionals abans i després de la secció crítica (preprotocol i postprotocol)
- Els protocols poden usar variables locals o globals però no són les de la regió crítica i viceversa

Algorithm 3.1: Critical section problem

global variables

p

local variables
 loop forever
 non-critical section
 preprotocol
 critical section
 postprotocol

q

local variables
 loop forever
 non-critical section
 preprotocol
 critical section
 postprotocol

- **La regió crítica ha de progressar:** Quan s'executen les sentències de la regió han d'acabar
- **La regió no crítica no necessita progressar:** Si el punter de control d'un procés és dins la seva regió no crítica el procés pot acabar o entrar dins un bucle infinit
- Quatre requisits de Dijkstra:
 1. Solució simètrica: No poden canviar el comportament o la prioritat estàtica dels processos
 2. No es poden fer suposicions de la velocitat relativa dels processos ni es pot suposar que siguin constants
 3. Entrada immediata o no interferència. Un procés que s'interromp no pot afectar als altres
 4. Processos que volen entrar simultàniament, la decisió de l'entrada ha de prendre un nombre finit de passes

Primer intent de solució

- La sentència *await turn=1* és una sentència que espera fins la condició *turn=1* es compleix i es pot implementar amb un bucle d'espera activa (*busy-wait*). Això no és desitjable degut al consum de CPU

Algorithm 3.2: First attempt	
integer turn $\leftarrow$ 1	
p	q
loop forever	loop forever
p1: non-critical section	q1: non-critical section
p2: await turn = 1	q2: await turn = 2
p3: critical section	q3: critical section
p4: turn $\leftarrow$ 2	q4: turn $\leftarrow$ 1

Correctesa del primer intent

- Exclusió mútua: *turn* només pot tenir un dels dos valors. Si p i q són a la SC *turn* val 1 i 2 a la vegada
- Progrés: Si p entra a la SC i en surt, *turn* val 2, i si després pretén tornar-hi entrar haurà d'esperar que q hi entri i en surti. Per tant està bloquejat!
- Espera limitada: Si q no entra a la SC es produeix una espera infinita
- Entrada immediata: Si *turn* val 2 i q està a la NSC p tampoc podrà entrar
- Sense suposicions de velocitat relativa: Si es suposa que els dos processos entren alternativament a la SC implica que la seva velocitat relativa és similar

Segon intent

- Problema anterior: Una única variable compartida pels dos processos
- Plantejament: Cada procés té la seva variable si *wanti* és vertader implica que el procés vol entrar a la seva SC. *await* assegura que el procés no entra mentre l'altre procés tingui la bandera posada

Algorithm 3.6: Second attempt	
boolean wantp ← false, wantq ← false	
p	q
loop forever	loop forever
p1: non-critical section	q1: non-critical section
p2: await wantq = false	q2: await wantp = false
p3: wantp ← true	q3: wantq ← true
p4: critical section	q4: critical section
p5: wantp ← false	q5: wantq ← false

- Aquest algorisme no garanteix l'exclusió mútua: pot passar que *wantp* i *wantq* siguin *true* ja que les sentències *await* no són atòmiques i per tant el procés pot ser interromput entre elles
- L'aparició d'escenaris prohibits garanteix la incorrectesa de l'algorisme

Algorithm 3.7: Second attempt (abbreviated)

boolean wantp $\leftarrow$ false, wantq $\leftarrow$ false

p

q

loop forever

p1: await wantq = false

p2: wantp $\leftarrow$ true

p3: wantp $\leftarrow$ false

loop forever

q1: await wantp = false

q2: wantq $\leftarrow$ true

q3: wantq $\leftarrow$ false

Process p	Process q	wantp	wantq
p1: await wantq=false	q1: await wantp=false	<i>false</i>	<i>false</i>
p2: wantp $\leftarrow$ true	q1: await wantp=false	<i>false</i>	<i>false</i>
p2: wantp$\leftarrow$true	q2: wantq $\leftarrow$ true	<i>false</i>	<i>false</i>
p3: wantp $\leftarrow$ false	q2: wantq$\leftarrow$true	<i>true</i>	<i>false</i>
p3: wantp $\leftarrow$ false	q3: wantq $\leftarrow$ false	<i>true</i>	<i>true</i>

Tercer intent

- Problema anterior: un procés verifica l'estat de l'altre abans de canviar el seu
- Plantejament: La sentència *await* és part de la SC per tant es canviara l'estat abans
- La solució verifica l'exclusió mútua i no es troba cap estat corresponent a cap escenari prohibit
- No se verifica el requisit de progrés ja que es pot arribar a una situació d'interbloqueig (*deadlock*)

Algorithm 3.8: Third attempt

boolean wantp $\leftarrow$ false, wantq $\leftarrow$ false

p

q

loop forever

loop forever

p1: non-critical section

q1: non-critical section

p2: wantp $\leftarrow$ true

q2: wantq $\leftarrow$ true

p3: await wantq = false

q3: await wantp = false

p4: critical section

q4: critical section

p5: wantp $\leftarrow$ false

q5: wantq $\leftarrow$ false

Process p

Process q

wantp

wantq

p1: non-critical section

q1: non-critical section

false

false

p2: wantp $\leftarrow$ true

q1: non-critical section

false

false

p2: wantp $\leftarrow$ true

q2: wantq $\leftarrow$ true

false

false

p3: await wantq=false

q2: wantq $\leftarrow$ true

true

false

p3: await wantq=false

q3: await wantp=false

true

true

Algorithm 3.8: Third attempt

boolean wantp $\leftarrow$ false, wantq $\leftarrow$ false

p

q

loop forever

p1: non-critical section
p2: wantp $\leftarrow$ true
p3: await wantq = false
p4: critical section
p5: wantp $\leftarrow$ false

loop forever

q1: non-critical section
q2: wantq $\leftarrow$ true
q3: await wantp = false
q4: critical section
q5: wantq $\leftarrow$ false

p2: wantp $\leftarrow$ true,
q2: wantq $\leftarrow$ true,
false, false

p3: await !wantq,
q2: wantq $\leftarrow$ true,
true, false

p3: await !wantq,
q3: await !wantp,
true, true

Deadlock

Quart intent

- Problema anterior: Quan un procés posa el seu *want a true* indica la intenció d'entrar a la SC i insisteix en el dret a fer-ho. *Deadlock*: Quan simultàniament **insisteixen en entrar**
- Plantejament: Si un procés sap que l'altre competeix per la SC assignarà al seu *want* un *false* immediatament seguit d'un *true*. Rompent l'interbloqueig

Algorithm 3.9: Fourth attempt

boolean wantp $\leftarrow$ false, wantq $\leftarrow$ false

p

q

loop forever

p1: non-critical section

p2: wantp $\leftarrow$ true

p3: while wantq

p4: wantp $\leftarrow$ false

p5: wantp $\leftarrow$ true

p6: critical section

p7: wantp $\leftarrow$ false

loop forever

q1: non-critical section

q2: wantq $\leftarrow$ true

q3: while wantp

q4: wantq $\leftarrow$ false

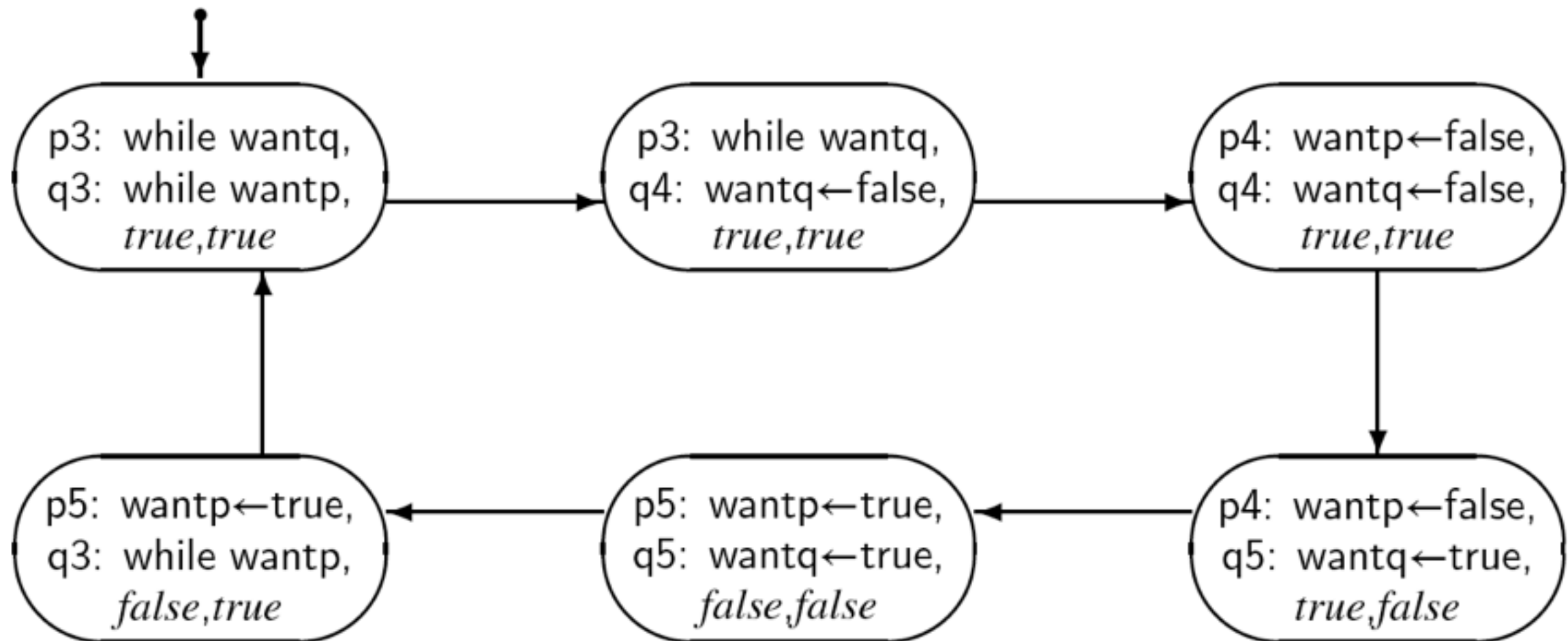
q5: wantq $\leftarrow$ true

q6: critical section

q7: wantq $\leftarrow$ false

- L'exclusió mútua està garantida: Si p entra abans, q es quedarà esperant, si hi ha competència i ambdós processos entren en el bucle per poder-ne sortir algun procés s'ha d'interrompre després de la sentència d'assignació false

- Pràctica i estadísticament no es produeixen esperes infinites però no es pot garantir que l'espera estarà limitada a un nombre de passos finit: **Bloqueig actiu** *livelock*



Algorisme de Dekker (1963)

- Solució com a combinació del primer i del quart intent
- En cas de competència la variable *turn* decidirà el procés amb dret d'entrar a la SC
- Si el procés *p* detecta competència per a la SC (*wantq*) consultarà el *turn* i si és seu (1) insistirà per entrar

Algorithm 3.10: Dekker's algorithm

boolean wantp $\leftarrow$ false, wantq $\leftarrow$ false
integer turn $\leftarrow$ 1

p

q

loop forever

loop forever

p1: non-critical section

q1: non-critical section

p2: wantp $\leftarrow$ true

q2: wantq $\leftarrow$ true

p3: while wantq

q3: while wantp

p4: if turn = 2

q4: if turn = 1

p5: wantp $\leftarrow$ false

q5: wantq $\leftarrow$ false

p6: await turn = 1

q6: await turn = 2

p7: wantp $\leftarrow$ true

q7: wantq $\leftarrow$ true

p8: critical section

q8: critical section

p9: turn $\leftarrow$ 2

q9: turn $\leftarrow$ 1

p10: wantp $\leftarrow$ false

q10: wantq $\leftarrow$ false

- Aquest algorisme satisfà l'exclusió mútua, no presenta interbloqueig ni espera infinita
- Si q vol entrar a la SC i p hi és, q es queda en espera. Quan p surti posarà el *turn* a 2 de manera que el següent entrar serà q malgrat p vulgui tornar entrar immediatament
- En cas de competència un procés i només un entrarà a la SC
- Quan un procés surti de la SC donarà el torn a l'altra com al principi

dekkerNoSync.c

```
9  volatile long count = 0;
10 volatile int estado[NUM_THREADS];
11 volatile int turno;

45     estado[0] = 0; //wantp = false
46     estado[1] = 0; //wantq = false
47     turno = 0;

13 void *counter(void *threadid) {
14     long tid, i, max = MAX_COUNT/NUM_THREADS;
15     tid = (long)threadid;
16     int otroHilo = (tid + 1) % NUM_THREADS; // Si ejecuta p és q i viceversa
17     for (i=0; i < max; i++) {
18         /* Dekker */
19         estado[tid] = 1;
20         while (estado[otroHilo]) { //while wanti
21             if (turno == otroHilo) {
22                 estado[tid] = 0;
23                 while (turno == otroHilo);
24                 estado[tid] = 1;
25             }
26         }
27         count++; //SC
28         turno = otroHilo;
29         estado[tid] = 0;
30     }
31     pthread_exit(NULL);
32 }
```

CounterDekker.java

```
public class CounterDekker implements Runnable {
```

```
    static final int THREADS = 2;
    static final int MAX_COUNT = 100000000;
    static volatile int estat[] = new int[THREADS];
    static volatile int torn = 0;
    static volatile int n = 0;
    int id;
```

```
    Thread[] threads = new Thread[THREADS];
    estat[0] = 0;
    estat[1] = 0;
    int i;
    for (i = 0; i < THREADS; i++) {
        threads[i] = new Thread(new CounterDekker(i));
        threads[i].start();
    }
    for (i = 0; i < THREADS; i++) {
        threads[i].join();
    }
```

```
    @Override
    public void run() {
        int max = MAX_COUNT / THREADS;
        int altreProces = (id + 1) % THREADS;
        System.out.println("Thread " + id);

        for (int i = 0; i < max; i++) {
            // Dekker
            estat[id] = 1;
            while (estat[altreProces] == 1) {
                if (torn == altreProces) {
                    estat[id] = 0;
                    while (torn == altreProces) {
                    }
                    estat[id] = 1;
                }
            }
            n += 1; //SC
            torn = altreProces;
            estat[id] = 0;
        }
    }
}
```

Algorisme de Peterson (1981)

- Solució basada en l'algorisme de Dekker junta les sentències *await* en una amb un condicional compost

Algorithm 3.13: Peterson's algorithm	
boolean wantp $\leftarrow$ false, wantq $\leftarrow$ false integer last $\leftarrow$ 1	
p	q
loop forever p1: non-critical section p2: wantp $\leftarrow$ true p3: last $\leftarrow$ 1 p4: await wantq = false or last = 2 p5: critical section p6: wantp $\leftarrow$ false	loop forever q1: non-critical section q2: wantq $\leftarrow$ true q3: last $\leftarrow$ 2 q4: await wantp = false or last = 1 q5: critical section q6: wantq $\leftarrow$ false

- Aquest algorisme satisfà l'**exclusió mútua**
- p i q mai poden estar a la SC a la vegada: Si p és a la SC $wantp$ val *true*. Al temps que:
 - $wantq$ és *false* (q acaba de sortir de la SC) ●
 - $last$ és 2 (q intenta entrar a la SC però espera) ●
 - q és a $q3$: (q intenta entrar a la SC després de posar $wantq$ a *true* però abans de posar $last$ a 2 i entrar en espera)
- Si els dos processos són a la SC verifiquen $wantp$ i $wantq$ i $last=1$ i $last=2$!!!

- Aquest algorisme no presenta **interbloqueig**:
 - Si hi ha competència el valor *last* indica el que continua
 - Si no hi ha competència el procés que vulgui entrar ho farà immediatament per què *wanti* per l'altre serà false
- No presenta **espera infinita** :
 - El procés que desitja entrar primer cedeix el torn abans de la comparació al bucle. Si després vol tornar entrar cedirà al lloc al que està esperant. Com a màxim l'espera és d'un torn

peterston.c

```
10 volatile long count = 0;
11 volatile int want[NUM_THREADS];
12 volatile int last;

14 void *counter(void *threadid) {
15     long id, i, max = MAX_COUNT/NUM_THREADS;
16     id = (long)threadid;
17     for (i=0; i < max; i++) {
18         //Peterson
19         int other = (id +1) % NUM_THREADS; // Si ejecuta p és q i viceversa
20         want[id] = 1;
21         last = id;
22         //__sync_synchronize();
23         while (want[other] && last == id)
24             //sched_yield()
25             ;
26         count++; //SC
27         want[id] = 0;
28     }
29     pthread_exit(NULL);
30 }

32 int main (int argc, char *argv[]) {
33     pthread_t threads[NUM_THREADS];
34     int rc;
35     long t;
36     want[0] = 0; //wantp = false
37     want[1] = 0; //wantq = false
38     last = 0;
39     for(t=0; t<NUM_THREADS; t++){
40         rc = pthread_create(&threads[t], NULL, counter, (void *)t);
```

CounterPeterson.java

```
static volatile int[] want = new int[THREADS];
static volatile int last = 0;
static volatile int n = 0;
int id;
```

@Override

```
public void run() {
    int max = MAX_COUNT / THREADS;
    int altreProces = (id + 1) % THREADS;
    System.out.println("Thread " + id);
    for (int i = 0; i < max; i++) {
        // Peterson
        want[id] = 1;
        last = id;
        while (want[altreProces] == 1 && last == id) {
        }
        System.out.println("Index: " + i + " Contador " + n);
        n += 1; //SC
        want[id] = 0;
    }
}
```

```
public static void main(String[] args) throws InterruptedException {
    Thread[] threads = new Thread[THREADS];
    want[0] = 0;
    want[1] = 0;

    int i;
    for (i = 0; i < THREADS; i++) {
        threads[i] = new Thread(new CounterPeterson(i));
        threads[i].start();
    }
}
```

- Aquests algorismes tot i ser correctes no funcionen en les arquitectures modernes que no assegurin la consistència seqüencial que es suposa a tots ells